



# Python for Fixed Income

COURSE 2 · BOND MATH

## Time Value of Money

WEEK 1 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# One idea under every bond price

**A payment due later is worth less than the same payment today.**

## The four bonds, settling 30 September 2026

| Issuer          | Rating | Coupon % | Years | Yield % |
|-----------------|--------|----------|-------|---------|
| Pellwick Paper  | A-     | 4.75     | 3     | 4.60    |
| Quorvane Energy | BBB    | 5.25     | 5     | 5.40    |
| Dunmarrow Rail  | BB+    | 6.125    | 7     | 6.125   |
| Halvering Foods | B+     | 7.50     | 10    | 7.70    |

Each pays a fixed coupon twice a year and repays 100 on 30 September of its last year.  
Today is a coupon date.

### Week 1: price and yield

- Day 1** Time value of money
- Day 2** Cash flows and price from yield
- Day 3** The price-yield relationship
- Day 4** Yield from price
- Day 5** Case study: a new issue calendar

# Your workspace, and the course's two units

The course repo: the notebook lives here



the setup cell

A fresh work folder in your temp folder: pfi\_course2\_01\_ and eight random characters

It holds the two Excel reference files, bondmath.py, and test\_bondmath.py. Everything the notebook writes goes here, and the course repo stays as it was.

Only the folder's name is printed: the full path has your user name in it.

| What          | Unit            | Example          |
|---------------|-----------------|------------------|
| Rates in      | percent a year  | yield_pct 5.40   |
| Money out     | per 100 of face | 76.6118 per 100  |
| Excel's rate  | a decimal       | 5.4/100 is 0.054 |
| One half-year | half the rate   | 0.054/2 is 0.027 |

**Every Excel formula in the course divides by 100 in plain sight: 5.4/100.**

# Future value: interest on interest

100 at 5.40 percent a year, compounded twice a year

| Half-year | Years | Value    |
|-----------|-------|----------|
| 1         | 0.5   | 102.7000 |
| 2         | 1.0   | 105.4729 |
| 4         | 2.0   | 111.2453 |
| 6         | 3.0   | 117.3337 |
| 8         | 4.0   | 123.7552 |
| 10        | 5.0   | 130.5282 |

Each half-year adds more than the last: 2.70 first, 3.4316 last.  
Simple interest would stop at 127.00.

```
future_value = amount * (1 + rate / frequency) ** (frequency * years)
100 * (1 + 0.054 / 2) ** 10
130.5282
```

## Excel side by side (tested in Excel)

`=FV(5.4/100/2,2*5,0,-100)`

returns 130.5282

`=A1*(1+5.4/100/2) in A2:A11`

A11 holds 130.5282

The growth column starts with 100 in A1. FV's arguments: rate per period, number of periods, payment each period (0: no coupons added), and the starting amount entered as -100. Money in and money out take opposite signs, so FV comes back positive.

# Present value and the discount factor

```
present_value = 100 / (1 + 0.054 / 2) ** 10
76.6118
discount factor: the present value of 1
```

Quorvane's 100, due 30 September 2031, is worth 76.6118 today at 5.40 percent. Grown for five years at the same rate, 76.6118 becomes 100 again.

**Excel side by side (tested in Excel)**

`=100/(1+5.4/100/2)^(2*5)`  
`=PV(5.4/100/2,2*5,0,100)`  
`=-PV(5.4/100/2,2*5,0,100)`

returns 76.6118  
returns -76.6118  
returns 76.6118

PV is negative: money received later is worth money paid now.

Each bond's 100 of redemption, at its own yield

| Issuer          | Years | Yield % | Factor | PV      |
|-----------------|-------|---------|--------|---------|
| Pellwick Paper  | 3     | 4.6     | 0.8725 | 87.2461 |
| Quorvane Energy | 5     | 5.4     | 0.7661 | 76.6118 |
| Dunmarrow Rail  | 7     | 6.125   | 0.6555 | 65.5527 |
| Halvering Foods | 10    | 7.7     | 0.4698 | 46.9753 |

A higher rate and a longer wait both push a present value down.  
Halvering has both.

# How often it compounds

Quorvane's 100, 5 years at 5.40 percent a year

| Frequency     | Periods | Future value | Present value |
|---------------|---------|--------------|---------------|
| 1, annual     | 5       | 130.0778     | 76.8771       |
| 2, semiannual | 10      | 130.5282     | 76.6118       |
| 4, quarterly  | 20      | 130.7600     | 76.4760       |

**0.2653**

per 100 of face: annual minus  
semiannual present value

**26,530.98**

dollars on 10,000,000 of face, from  
the frequency alone

**A yield is complete only with its frequency.**

## Excel side by side (tested in Excel)

|                                        |                  |
|----------------------------------------|------------------|
| <code>=FV(5.4/100/1,1*5,0,-100)</code> | returns 130.0778 |
| <code>=PV(5.4/100/1,1*5,0,100)</code>  | returns -76.8771 |
| <code>=FV(5.4/100/4,4*5,0,-100)</code> | returns 130.7600 |
| <code>=PV(5.4/100/4,4*5,0,100)</code>  | returns -76.4760 |

The frequency appears twice in each formula: in the rate and in the number of periods. Change one and not the other, and Excel computes a different number without complaint.

**The wrong frequency is a convention error: the code runs and the number looks reasonable. Today's AI check is one.**

# bondmath.py: the docstring comes first

```
def present_value(amount, rate_pct, years, frequency=2):  
    """Discount an amount due in `years` at an annual rate  
    compounded `frequency` times a year.  
  
    Units: amount per 100 of face, rate_pct in percent.  
    Convention: amount / (1 + rate / frequency) ** (frequency x years).  
    Excel: =-PV(rate_pct/100/frequency, frequency*years, 0, amount)  
    """  
    if frequency not in (1, 2, 4):  
        raise ValueError(...)  
    period_rate = rate_pct / 100 / frequency  
    return amount / (1 + period_rate) ** (frequency * years)
```

Shortened for the slide. The notebook shows the full function.

One or two functions a day. By week 3 the module prices bonds and measures their risk.

## The docstring states, first

1. the units, in and out
2. the convention: compounding
3. the matching Excel formula

**The docstring is the request: to a colleague, or to an AI assistant.**

# pytest: a pass and a failure

```
def test_present_value_matches_excel_pv():
    for row in tvml_rows("PV"):
        result = bondmath.present_value(...)
        # Excel's PV is negative
        assert result == pytest.approx(
            -float(row["excel_value"]), abs=1e-6)
```

A test is an assert kept in a file. pytest finds every test\_ function in every test\_ file, runs them, and reports. The expected numbers come from Excel, never from the code being tested.

**In your own terminal:** `python -m pytest`

## A pass

```
... [100%]
3 passed
```

## A failure: the test, not the function

```
E  assert 76.61178196895271
      == -76.61178196895273
1 failed
```

Read from the bottom up. Same number, opposite sign: the colleague's test forgot Excel's sign convention. A failure says two things disagree. Which is wrong is your call.

# Excel is the answer key

| Job                           | Excel formula (tested)    | Excel returns |
|-------------------------------|---------------------------|---------------|
| Grow 100, 5 years, semiannual | =FV(5.4/100/2,2*5,0,-100) | 130.5282      |
| The same, written out         | =100*(1+5.4/100/2)^(2*5)  | 130.5282      |
| Discount 100 due in 5 years   | =PV(5.4/100/2,2*5,0,100)  | -76.6118      |
| The same, written out         | =100/(1+5.4/100/2)^(2*5)  | 76.6118       |
| Annual compounding            | =PV(5.4/100/1,1*5,0,100)  | -76.8771      |
| Quarterly compounding         | =PV(5.4/100/4,4*5,0,100)  | -76.4760      |

36

FV and PV rows in the reference file,  
each compared with bondmath

1.4e-09

the largest gap, against a tolerance of  
0.000001 per 100

Every function you write is checked against a number Excel produced. The test file does it on every run.

# Git: the undo button, kept away from the course repo

The course repo: the notebook runs here. Git never acts on it from the notebook.

↓  
-C "{work}" on every Git line

The work folder: Git keeps its record here, in a hidden .git folder.

The guard asks Git for the top of the repository and stops the notebook unless it is the work folder.

**In your own terminal, in your own folder, type the same commands without -C: plain git status.**

```
!git -C "{work}" init -q -b main
!git -C "{work}" config user.name "Course Student"
!git -C "{work}" status
!git -C "{work}" add bondmath.py test_bondmath.py
!git -C "{work}" commit -m "Add future_value ..."
```

```
[main (root-commit) 1a2b3c4] Add future_value ...
2 files changed, 109 insertions(+)
```

| Word      | Meaning                         |
|-----------|---------------------------------|
| commit    | a saved version, with a message |
| staged    | chosen for the next commit      |
| untracked | seen by Git, not recorded       |

# Bring it home: your own my\_bondmath folder

1

## Make the folder

my\_bondmath, in your projects folder beside the course repo. Never just bondmath.

2

## Copy the data

The two reference CSVs, from the course repo's data folder.

3

## Today's files

bondmath.py and test\_bondmath.py, from the work folder the notebook printed.

4

## Run the tests

python -m pytest -q, and see 3 passed.

5

## First commit

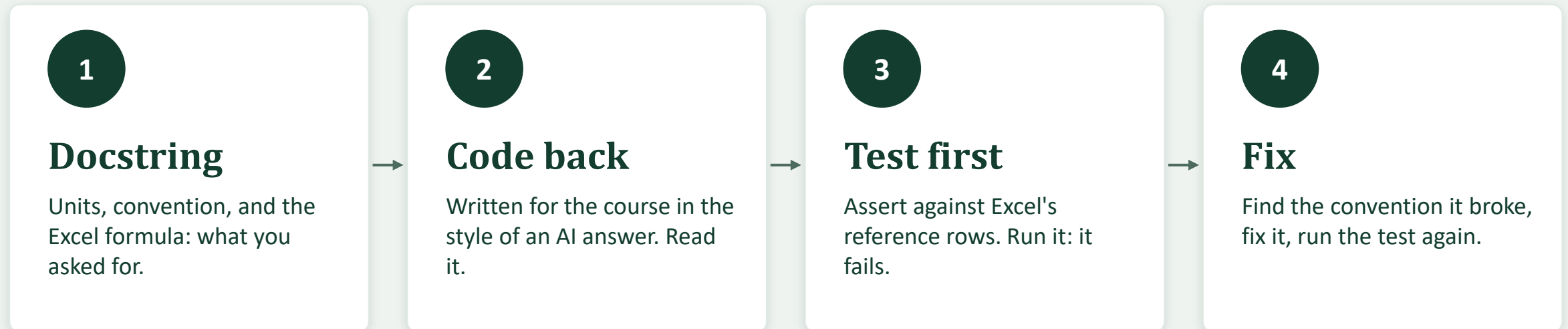
git init, your name and email, git add, git commit.

**No notebook ever touches my\_bondmath. From tomorrow you paste each day's new functions and tests, run the tests, read git diff, and commit.**

In Colab: download bondmath.py and test\_bondmath.py from the file browser. Git blocked at work? Read the saved output and do step 5 at home.

# The AI check

The docstring is the prompt. The code that comes back runs, and has one convention error.



Today: `present_value` compounds once a year, when the docstring says twice.

*Code that runs is not the same as code that is right.*

# What you can do now

- Grow an amount and discount a payment, with periodic compounding
- Say why the frequency matters, and spot the wrong one
- Write a function docstring first: units, convention, Excel formula
- Test it against Excel with pytest, and read a pass and a failure
- Commit it with Git, and keep your own copy in my\_bondmath

**Week 1, Day 2** Cash flows and price from yield: a bond is a list of payments, each with its own discount factor